

Hardware interface with ROS

Prof. Muthukumar

Working with ROS and Core/Controller Board

- Programming the STM32 microcontroller
 - Use STM32Cube IDE
 - Tutorials on STM32CubeIDE
 - Introduction to STM32CubeMX
 - Working with the STM32CubeIDE ioc files.
- Programming the STM32
 - Use STM32CubeProgrammer using USB (or)
 - Use third party tools – through bootloader

Working with roserial

- roserial
 - **roserial** is a protocol and library designed for integrating microcontrollers and embedded systems with the **Robot Operating System (ROS)**. It provides a method for communication between ROS nodes and devices like Arduino, STM32, or other low-level controllers that don't have the resources to run a full ROS environment.

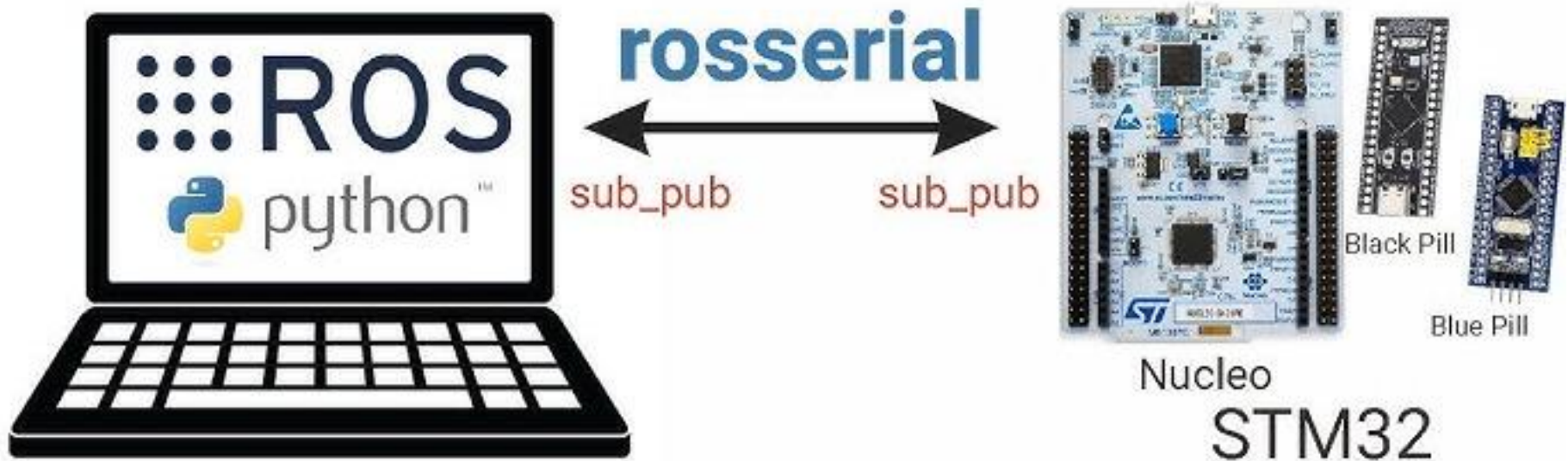
roserial_arduino	support for Arduino compatible boards including UNO, Leonardo, MEGA, DUE, Teensy 3.x and LC, Spark,  STM32F1 ,  STM32Duino ,  ESP8266 and ESP32
roserial_embeddedlinux	support for Embedded Linux (eg, routers)
roserial_windows	support for communicating with Windows applications
roserial_mbed	support for mbed platforms
roserial_tivac	support for TI's Launchpad boards, TM4C123GXL and TM4C1294XL
roserial_vex_v5	support for VEX V5 Robot Brain
roserial_vex_cortex	support for VEX Cortex board
 roserial_stm32	support for STM32 MCUs, based on STM32CubeMX HAL
 ros-teensy	support for teensy platforms

Rosserial Server side

- ROS-side Interfaces
 - Devices running roserial code require a node on the host machine to bridge the connection from the serial protocol to the more general ROS network:

`roserial_python` A Python-based implementation (recommended for PC usage).

`roserial_server` A C++ implementation based on [the ShapeShifter message](#), some limitations compared to `roserial_python` but recommended for high-performance applications.

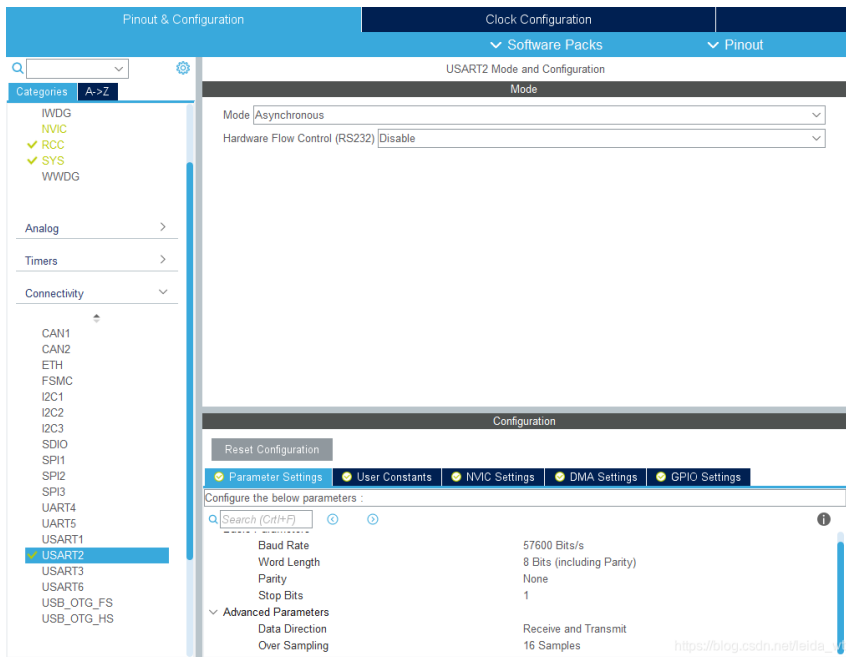


Include ros_lib in STM32CubeIDE

- Install roserial
 - `sudo apt-get install ros-noetic-roserial`
 - `sudo apt-get install ros-noetic-roserial-arduino`
- Clone this repository in your workspace and make
 - https://github.com/yoneken/roserial_stm32
- Run the following command in the Core/Inc folder
 - `roslaunch roserial_stm32 make_libraries.py .`
- Reference the Core/Inc/ros_lib folder
- Modify some files in the ros_lib folder (see video)
- Change the project to C++ project

STM32cubeIDE – Add UART1

- Add UART1 with DMA in your ioc file



Parameter Settings	User Constants	NVIC Settings	DMA Settings	GPIO Settings
NVIC Interrupt Table				
		Enabled	Preemption Priority	Sub Priority
DMA1 stream5 global interrupt		☒	0	0
DMA1 stream6 global interrupt		☒	0	0
USART2 global interrupt		☒	0	0

https://blog.csdn.net/leida_wl

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

DMA Request	Stream	Direction	Priority
USART2_RX	DMA1 Stream 5	Peripheral To Memory	High
USART2_TX	DMA1 Stream 6	Memory To Peripheral	High

Add

Delete

https://blog.csdn.net/leida_wl

```
#include <main.hpp>

/* Private includes ----- */
/* USER CODE BEGIN Includes */
#include <ros.h>
#include <std_msgs/String.h>

/* USER CODE BEGIN 0 */
ros::NodeHandle nh;

std_msgs::String str_msg;
ros::Publisher chatter("chatter", &str_msg);
char hello[] = "Hello world!";

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart){
nh.getHardware()->flush();
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
nh.getHardware()->reset_rbuf();
}
```

```
void setup(void)
{
  nh.initNode();
  nh.advertise(chatter);
}
```

```
void loop(void)
{
```

```
  HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
```

```
  str_msg.data = hello;
  chatter.publish(&str_msg);
```

```
  nh.spinOnce();
```

```
  HAL_Delay(1000);
}
```

```
/* USER CODE BEGIN 2 */
setup();
/* USER CODE END 2 */
```

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */
```

```
/* USER CODE BEGIN 3 */
loop();
}
```


Subscriber example

```
// Subscriber:
void led_cb(const std_msgs::Empty &msg) {
    HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_4);
}
ros::Subscriber<std_msgs::Empty> led_sub("toggle_led", &led_cb);

// Setup node:
void setup(void) {
    nh.initNode();
    nh.advertise(chatter);
    nh.subscribe(led_sub);
}
```

HC-SR04 Sensor

- sensor_msgs/Range Message

```
# Single range reading from an active ranger that emits energy and reports
# one range reading that is valid along an arc at the distance measured.
# This message is not appropriate for laser scanners. See the LaserScan
# message if you are working with a laser scanner.
```

```
# This message also can represent a fixed-distance (binary) ranger. This
# sensor will have min_range==max_range==distance of detection.
# These sensors follow REP 117 and will output -Inf if the object is detected
# and +Inf if the object is outside of the detection range.
```

```
Header header          # timestamp in the header is the time the ranger
                        # returned the distance reading
```

```
# Radiation type enums
# If you want a value added to this list, send an email to the ros-users list
uint8 ULTRASOUND=0
uint8 INFRARED=1
```

```
uint8 radiation_type    # the type of radiation used by the sensor
                        # (sound, IR, etc) [enum]
```

```
float32 field_of_view   # the size of the arc that the distance reading is
                        # valid for [rad]
                        # the object causing the range reading may have
                        # been anywhere within -field_of_view/2 and
                        # field_of_view/2 at the measured range.
                        # 0 angle corresponds to the x-axis of the sensor.
```

```
float32 min_range       # minimum range value [m]
float32 max_range       # maximum range value [m]
                        # Fixed distance rangers require min_range==max_range
```

```
float32 range           # range data [m]
                        # (Note: values < range_min or > range_max
                        # should be discarded)
                        # Fixed distance rangers only output -Inf or +Inf.
                        # -Inf represents a detection within fixed distance.
                        # (Detection too close to the sensor to quantify)
                        # +Inf represents no detection within the fixed distance.
                        # (Object out of range)
```

IMU

- sensor_msgs/Imu Message

```
# This is a message to hold data from an IMU (Inertial Measurement Unit)
#
# Accelerations should be in m/s^2 (not in g's), and rotational velocity should be in rad/sec
#
# If the covariance of the measurement is known, it should be filled in (if all you know is the
# variance of each measurement, e.g. from the datasheet, just put those along the diagonal)
# A covariance matrix of all zeros will be interpreted as "covariance unknown", and to use the
# data a covariance will have to be assumed or gotten from some other source
#
# If you have no estimate for one of the data elements (e.g. your IMU doesn't produce an orientation
# estimate), please set element 0 of the associated covariance matrix to -1
# If you are interpreting this message, please check for a value of -1 in the first element of each
# covariance matrix, and disregard the associated estimate.
```

Header header

```
geometry_msgs/Quaternion orientation
float64[9] orientation_covariance # Row major about x, y, z axes

geometry_msgs/Vector3 angular_velocity
float64[9] angular_velocity_covariance # Row major about x, y, z axes

geometry_msgs/Vector3 linear_acceleration
float64[9] linear_acceleration_covariance # Row major x, y z
```

Rosserial References

- Rosserial
 - <http://wiki.ros.org/rosterial>
- STM32duino
 - <https://github.com/stm32duino>
- Rosserial_Arduino_Library (stm32 Arduino)
 - https://github.com/frankjoshua/rosterial_arduino_lib
- Rosserial_stm32 (stm32 HAL)
 - https://github.com/yoneken/rosterial_stm32
- Rosserial USB_VCP
 - https://github.com/ltamare4/ROS_stm32f1_rosterial_USB_VCP

Demo